

DPDP COMPLIANCE AUDIT

taskmaster

Digital Personal Data Protection Act 2023



Grade B — Adequate

Scanned: 08 April 2026

Files indexed: 18

Languages: Node.js, Express, Mongoose, React, Next.js, JWT

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

The application collects user name, email, and password for authentication and task management. The primary risk lies in the potential for unauthorized access to user accounts and the sensitive data they manage, especially if the JWT secret is compromised or if there are vulnerabilities in the password hashing mechanism.

Total findings: 14

HIGH: 3

MEDIUM: 2

INFO: 3

PASS: 5

Tech Stack: Node.js, Express, Mongoose, React, Next.js

Risk Profile: The application collects user name, email, and password for authentication and task management. The primary risk lies in the potential for unauthorized access to user accounts and the sensitive data they manage, especially if the JWT secret is compromised or if there are vulnerabilities in the password hashing mechanism.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	14/22 (64%)	WARN
Section 6(6)	8	8/8 (100%)	PASS
Section 8(1)	18	18/18 (100%)	PASS
Section 8	10	7.0/10 (70%)	WARN
Section 8(3)	8	4.0/8 (50%)	WARN
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	8/8 (100%)	PASS
Section 9	6	6/6 (100%)	PASS
Section 11	6	6/6 (100%)	PASS
Section 16	4	4/4 (100%)	PASS
Section 5	2	2/2 (100%)	PASS
Section 13 — Grievance Redressal		0/4 (0%)	FAIL

Estimated Total Remediation Effort: 1.9–7.0 engineering days

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
INCOMPLETE_DELETION_COVERAGE	Section 8 — Data Principal Rights	MEDIUM	N/A

RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	backend/models/User.js
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	frontend/src/context/AuthContext.tsx
CONSENT_PRESENT	Section 6 — Consent	PASS	frontend/src/app/dashboard/page.tsx
DELETION_MECHANISM_PRESENT	Section 8 — Data Principal Rights	PASS	N/A
BREACH_INDICATORS_ADEQUATE	Section 8(6) — Breach	PASS	N/A
DATA_ACCESS_PRESENT	Section 11 — Right to Access	PASS	frontend/src/app/dashboard/page.tsx

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (9–32 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- Audit Trail Missing (4–12 hrs)
- Grievance Officer Missing (1–4 hrs)

Week 2 — High Priority (4–16 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- Incomplete Deletion Coverage (2–8 hrs)
- Retention Missing — models/User.js (2–8 hrs)

Week 4 — Advisory (2–8 hours)

Address before next compliance review.

- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 15–56 hours (1.9–7.0 engineering days)

Details

Finding 1 of 14

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The absence of a consent mechanism means personal data may be processed without explicit user agreement, violating data subject rights and potentially leading to non-compliance penalties.

Fix:

1. Implement a consent management system (e.g., using a library or custom solution) to obtain and record user consent for data processing activities.
2. Integrate consent checks into API endpoints and frontend components where personal data is collected or processed.
3. Add a clear privacy policy link and a dedicated section for managing consent preferences in the user interface.
4. Ensure consent is granular and can be withdrawn by the user at any time.

DPDP:

Section 6 — Consent

Example:

```
import React, { useState } from 'react'; function ConsentBanner() { const [consentGiven, setConsentGiven] = useState(false); const handleAccept = () => { // Logic to record consent (e.g., API call, set cookie) setConsentGiven(true); }; if (consentGiven) return null; return ( <div style={{ position: 'fixed', bottom: 0, width: '100%', backgroundColor: '#f0f0f0', padding: '10px', textAlign: 'center' }}> We use cookies to improve your experience. By continuing to use our site, you agree to our use of cookies. <button onClick={handleAccept}>Accept</button> </div> ); }
```

Finding 2 of 14

[Section 8(4) — Audit Trail]

2. [HIGH] AUDIT_TRAIL_MISSING — N / A:10**Evidence snippet:** Defines routes for task operations, all protected by 'protect'.**Estimated fix time:** 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The absence of an audit trail means there are no records of who accessed or modified personal data, making it impossible to investigate breaches or ensure accountability as required by DPDP.

Fix:

1. Implement a centralized logging mechanism (e.g., using Winston or Pino in Node.js) to capture relevant events.
2. Define a consistent log format that includes timestamp, user ID, action performed, and affected data.
3. Create middleware or decorators in your backend framework (Express.js) to automatically log requests and data modifications.
4. Ensure logs are stored securely and retained according to policy, and consider a separate audit log database.

DPDP:

Section 8(4) — Audit Trail

Example:

```
const express = require('express'); const app = express(); const logger = require('./logger');
// Assume a logger setup app.use((req, res, next) => { const start = Date.now();
res.on('finish', () => { const duration = Date.now() - start; logger.info(`${req.method}
${req.originalUrl} - ${res.statusCode} (${duration}ms) - User: ${req.user ? req.user.id :
'guest'}`); }); next(); }); // Example protected route app.get('/protected-data', (req, res)
=> { // Log specific data access if needed logger.audit('Accessed protected data by user
${req.user.id}`); res.send('Sensitive data'); });
```

Finding 3 of 14

[Section 13 — Grievance Redressal]

3. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A**Estimated fix time:** 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route ****and**** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The absence of a designated Grievance Officer or Data Protection Officer (DPO) contact point means individuals have no clear channel to raise concerns or exercise their data protection rights, hindering effective redressal and compliance.

Fix:

1. Designate a Grievance Officer or DPO responsible for handling data protection inquiries and complaints.
2. Publish the contact details (email address or a dedicated contact form URL) of the Grievance Officer/DPO prominently on your website, typically in the footer or a dedicated 'Contact Us' or 'Privacy' page.
3. Create a dedicated page or section on your website that outlines the grievance redressal process and provides the contact information.
4. Ensure the contact information is accurate and regularly reviewed.

DPDP:

Section 13 — Grievance Redressal

Example:

```
<!-- Example: Footer snippet in a React/Next.js app --> <footer className="bg-gray-800 text-white p-4"> <div className="container mx-auto text-center"> <p>&copy; 2023 Your Company. All rights reserved.</p> <p> For any privacy concerns or to exercise your data rights, please contact our Data Protection Officer: <a href="mailto:dpo@example.com" className="text-blue-300 hover:underline"> dpo@example.com</a> </p> <p> <a href="/privacy" className="text-blue-300 hover:underline">Privacy Policy</a> | <a href="/terms" className="text-blue-300 hover:underline">Terms of Service</a> </p> </div> </footer>
```

Finding 4 of 14

[Section 8 — Data Principal Rights]

4. [MEDIUM] INCOMPLETE_DELETION_COVERAGE — N / A**Estimated fix time:** 2–8 hours

Deletion mechanism detected but may not cover all PII storage. Uncovered files: backend/config/db.js, backend/models/Task.js, backend/models/User.js. DPDP Section 8 requires complete erasure of all personal data across all systems when a user requests deletion.

Risk:

Incomplete deletion coverage means personal data might persist in the system after a deletion request, violating the right to erasure and potentially leading to unauthorized data retention.

Fix:

1. Implement a clear and comprehensive data deletion process for all types of personal data stored by the application.
2. Ensure that deletion requests trigger the removal of data from all relevant databases, caches, and backups (where feasible and legally permissible).
3. Provide a mechanism for users to request the deletion of their personal data, either through their account settings or a dedicated request form.
4. Log all deletion requests and their successful execution to maintain an audit trail.

DPDP:

Section 8 — Data Principal Rights

Example:

```
async function deleteTask(req, res) { try { const taskId = req.params.id; // Assume 'Task' is
your Mongoose model const task = await Task.findByIdAndDelete(taskId); if (!task) { return
res.status(404).json({ message: 'Task not found' }); } // Log the deletion event
console.log(`Task ${taskId} deleted by user ${req.user.id}`); res.json({ message: 'Task
deleted successfully' }); } catch (error) { res.status(500).json({ message: 'Server error
deleting task' }); } }
```

Finding 5 of 14

[Section 8(3) — Retention]

5. [MEDIUM] RETENTION_MISSING — backend / models / User.js:15**Evidence snippet:** The userSchema defines fields like 'name', 'email', and 'password', which are considered PII.**Estimated fix time:** 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

The User model contains Personally Identifiable Information (PII) but lacks any defined data retention or expiry periods, potentially leading to indefinite storage of personal data.

Fix:

1. Add a 'retentionPeriod' field (e.g., Number or String) to userSchema in backend/models/User.js to specify how long user data should be kept.
2. Implement a background job or scheduled task that periodically checks for and deletes user data exceeding their defined retention period.
3. Consider adding a 'deletedAt' timestamp field to userSchema to soft-delete records before permanent removal.

DPDP:

Section 8(3) — Retention

Example:

```
const userSchema = new mongoose.Schema({ name: { type: String, required: true }, email: { type: String, required: true, unique: true }, password: { type: String, required: true }, retentionPeriod: { type: String, // e.g., '1y', '6m', '30d' required: false // Or true if mandatory for all users }, { timestamps: true } });
```

Finding 6 of 14

[Section 11 — Data Portability]

6. [LOW] DATA_PORTABILITY_MISSING — N / A:6**Evidence snippet:** The getUserProfile endpoint is defined, which retrieves user data.**Estimated fix time:** 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The application does not provide an endpoint or mechanism for users to request and receive their personal data in a machine-readable format, failing to meet DPDP data portability requirements.

Fix:

1. Create a new API endpoint in backend/routes/userRoutes.js (e.g., '/export-data') protected by the 'protect' middleware.
2. Implement a controller function for this endpoint that fetches the authenticated user's data from the database.
3. Format the user's data into a standard machine-readable format like JSON and return it as a file download.
4. Add a button or link in the frontend (e.g., in user profile settings) that triggers this new export endpoint.

DPDP:

Section 11 — Data Portability

Example:

```
// backend/routes/userRoutes.js router.get('/export-data', protect, exportUserData); //
backend/controllers/userController.js const exportUserData = async (req, res) => { try { const
user = await User.findById(req.user.id).select('-password'); // Exclude password
res.setHeader('Content-Type', 'application/json'); res.setHeader('Content-Disposition',
'attachment; filename="user_data.json"'); res.status(200).send(JSON.stringify(user, null, 2));
} catch (error) { res.status(500).json({ message: 'Failed to export data' }); } };
```

Finding 7 of 14

[Section 5 — Notice]

7. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 8 of 14

[Section 5 — Purpose Limitation]

8. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 9 of 14

[Section 16 — Cross-Border Transfer]

9. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 10 of 14

[Section 6 — Consent]

10. [PASS] CONSENT_PRESENT — frontend / src / context / AuthContext.tsx

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 11 of 14

[Section 6 — Consent]

11. [PASS] CONSENT_PRESENT — frontend / src / app / dashboard / page.tsx

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 12 of 14

[Section 8 — Data Principal Rights]

12. [PASS] DELETION_MECHANISM_PRESENT — N / A

Data deletion mechanism detected. Found 0 deletion endpoint(s) and 2 deletion signal(s). Note: Verify deletion cascades to all PII tables (sessions, logs, related records).

Fix:

1. Verify deletion cascades to: sessions, audit_logs, third-party sync records.
2. Use database CASCADE DELETE or explicit multi-table deletion in deletion service.
3. Test deletion completeness: create account, delete, query all PII tables for orphaned data.

Finding 13 of 14

[Section 8(6) — Breach]

13. [PASS] BREACH_INDICATORS_ADEQUATE — N / A

Logging and/or breach alerting present.

Finding 14 of 14

[Section 11 — Right to Access]

14. [PASS] DATA_ACCESS_PRESENT — frontend / src / app / dashboard / page.tsx

Data access endpoint detected for authenticated users.

AI-Identified Compliance Gaps

The following gaps were identified by AI analysis and cross-validated. These are advisory observations, not verified rule violations. Human review is required before acting on these findings.

[AI-INFERRED] Incomplete user data access endpoint

Confidence: 60%

Section 8 Right to Access

The ``getUserProfile`` function in ``backend/controllers/userController.js`` retrieves user data (name, email) but does not appear to include all PII stored for the user, such as potentially other profile fields not explicitly listed in the `data_flows` but present in the User model.

Recommendation: Modify the `getUserProfile` function in `backend/controllers/userController.js` to ensure it retrieves and returns all PII fields associated with the user from the User model, aligning with Section 8 of DPDP.

[AI-INFERRED] No age verification for new users

Confidence: 40%

Section 9 Children

The user registration process (POST /users) in `backend/controllers/userController.js` does not include any mechanism to collect or verify the age of users, which is a requirement under Section 9 if the service is intended for or accessible by children.

Recommendation: Implement age verification logic within the user registration endpoint (POST /users) in `backend/controllers/userController.js`. This could involve adding an age field and validation, or a checkbox for age confirmation.

[AI-INFERRED] Missing audit trail for data access

Confidence: 65%

Section 8(4) Audit Trail

While the `AUDIT\_TRAIL\_MISSING` rule fired, the specific implementation details for logging personal data access events are not evident. The `getUserProfile` function in `backend/controllers/userController.js` accesses user PII but lacks explicit logging of this access event.

Recommendation: Add logging within the `getUserProfile` function in `backend/controllers/userController.js` to record when a user's personal data is accessed, including the user ID, timestamp, and the type of data accessed, to comply with Section 8(4).

Deep Compliance Review

Overall Risk: HIGH

The application exhibits several cross-cutting data protection weaknesses, particularly concerning the handling of Personally Identifiable Information (PII). Findings across multiple layers indicate insufficient purpose limitation for PII, potential PII exposure through logging and tokenization, and inconsistent secure handling of sensitive data like passwords. These issues collectively increase the risk of unauthorized access and data misuse, necessitating immediate architectural attention.

Estimated remediation: 17 engineering day(s)

Most Critical Action: Implement explicit purpose limitation for all PII fields ('name', 'email') in the User data model and enforce data minimization by removing unnecessary fields from API responses and frontend state.

Layer Breakdown

Configuration & Secrets — 0 finding(s). The configuration files reviewed (eslint, next.config, postcss, tailwind) do not contain any direct DPDP compliance issues. They are standard configuration files for a Next.js frontend project and do not handle sensitive data or implement specific data protection controls.

Data Models & Storage — 2 finding(s). The data models for User and Task are straightforward Mongoose schemas. The User model stores PII (name, email, hashed password). The Task model references the User, indicating a relationship where tasks are associated with specific users. The primary compliance concern at this layer is the handling and purpose of PII stored in the User model, as well as ensuring that data minimization principles are applied.

Authentication & Session — 2 finding(s). The authentication layer correctly uses JWT for stateless authentication. However, there are minor risks related to the logging of errors in the backend and the storage of tokens in `localStorage` on the frontend, which could be improved for better data protection.

Third-Party & PII Flows — 2 finding(s). This layer handles user registration, login, and profile updates. A key concern is the inclusion of user identifiers in JWT tokens and the transmission of passwords in API requests. While HTTPS is assumed for transport security, the application logic itself could be strengthened to further minimize PII exposure and ensure data integrity during transit.

API Routes & Controllers — 3 finding(s). The API routes and controllers layer shows minor compliance risks related to the handling of sensitive data, specifically passwords, in the frontend profile update functionality. There's also a potential for PII exposure through console logging and an unnecessary inclusion of a password field in the profile data state. These issues are primarily related to data minimization and secure handling principles.